

Problem Set 6 Lab Report: Word Games in Python

Overview

Problem Set 6 focused on creating two word-based games using Python. The first game was a one-player Scrabble-style game in which a player attempts to create valid words from a randomly generated hand of letters. The second game was a two-player game called Ghost, where players alternately add letters to a growing word fragment while attempting to avoid completing a valid word or creating an invalid fragment.

The purpose of the assignment was to strengthen programming skills involving string manipulation, file handling, dictionaries, loops, conditional logic, functions, and user interaction. Both games relied on a large dictionary file containing valid English words and demonstrated practical applications of searching and validating textual data.

Problem One: Scrabble-Style Word Game

The first program implemented a simplified Scrabble-style game. The game loaded a dictionary of over 83,000 words from a text file and generated a random hand of seven letters containing both vowels and consonants. The player attempted to create valid words using only the letters available in the hand.

Several functions were used to modularize the program. These functions handled tasks such as loading the dictionary, generating hands, displaying available letters, validating words, updating the hand after a word was played, and calculating word scores. Each letter was assigned a point value similar to traditional Scrabble scoring.

The game repeatedly prompted the user to enter words until the hand was exhausted or the player chose to end the round. Valid words earned points based on their letter values, while invalid words were rejected. After a valid word was accepted, the letters used were removed from the hand.

Testing demonstrated that valid words were correctly recognized, invalid words were rejected, scores were calculated properly, and letters were removed from the hand after use. For example, the words “run” was successfully accepted during testing and produced the correct score calculations.

```
Loading word list from file...
83667 words loaded
Enter n for new hand or e to end game: n
Current hand:
e u r v p n q
Given the letters in your hand, make a word or enter '.' to end: run
The word run got you 3 points. Total score: 3
Current hand:
e v p q
Given the letters in your hand, make a word or enter '.' to end: ev
Invalid word; please try again.
Current hand:
e v p q
Given the letters in your hand, make a word or enter '.' to end: pev
Invalid word; please try again.
Current hand:
e v p q
Given the letters in your hand, make a word or enter '.' to end: qev
Invalid word; please try again.
Current hand:
e v p q
Given the letters in your hand, make a word or enter '.' to end: evp
Invalid word; please try again.
Current hand:
e v p q
Given the letters in your hand, make a word or enter '.' to end: .
Final score: 3
Enter n for new hand or e to end game: e
Thanks for playing!
```

Problem Two: Ghost Word Game

The second program implemented the game, Ghost. In this game, two players take turns entering letters that are appended to a growing word fragment. The objective is to avoid completing a valid word and avoid creating a fragment that cannot begin any valid dictionary word.

The program loaded the same dictionary file used in the first game and maintained a running fragment. After each letter was entered, the program checked two conditions. First, it determined whether the fragment had become a complete word of at least four letters. Second, it verified whether the fragment still matched the beginning of at least one valid word in the dictionary.

If a player completed a valid word, that player lost the game. Similarly, if a player created a fragment that could not begin any valid dictionary word, that player also lost. The game continued alternating between Player 1 and Player 2 until one of the losing conditions occurred.

The Ghost game demonstrated how prefixes and string searching can be used in practical applications. The solution required validating partial strings against an external word list and managing game state across multiple turns.

```
PS C:\Users\alanp\AppData\Local\Programs\Microsoft VS Code> & C:\Users\alanp\AppData\Local\Microsoft\WindowsApps\python3.13.exe "//TRUENAS/ShadowRealm/Alan's Prometheus Backup/Education/US College/University of the Cumberland/MSDS-532/assignment6_2.py"

Current fragment:
Player 1, enter a letter: A

Current fragment: a
Player 2, enter a letter: c

Current fragment: ac
Player 1, enter a letter: e

Current fragment: ace
Player 2, enter a letter: d

aced is a complete word.
Player 2 loses.
PS C:\Users\alanp\AppData\Local\Programs\Microsoft VS Code> █
```

Programming Concepts Demonstrated

This assignment incorporated several important Python programming concepts. File handling was used to load the dictionary file. Functions were used extensively to create modular and reusable code. Dictionaries were used to track letter frequencies and scoring values. Strings were manipulated throughout both games to validate words and manage user input.

The assignment also reinforced the use of loops and conditional statements. Both games relied on repeated user interaction and decision-making based on player input. Additionally, the assignment demonstrated the importance of breaking larger problems into smaller functions, which improved readability and maintainability.

Another important concept was data validation. Both games required checking whether user input satisfied specific rules before allowing the program to continue. This type of validation is common in real-world software applications.

Testing

Multiple tests were performed to verify the functionality of both programs. In the Scrabble-style game, testing confirmed that valid words were accepted, invalid words were rejected, scores were calculated correctly, and used letters were removed from the hand. The game also correctly tracked cumulative scores and terminated properly when the player entered a period.

In the Ghost game, testing confirmed that fragments were validated against the dictionary and that losing conditions were correctly detected. The program successfully identified when a complete word had been formed and when a fragment no longer matched any dictionary entry.

These tests demonstrated that both programs behaved as expected and satisfied the assignment requirements.

Conclusion

Problem Set 6 provided practical experience developing interactive word games using Python. The Scrabble-style game demonstrated dictionary lookups, scoring systems, and inventory management through letter tracking. The Ghost game demonstrated string searching, prefix validation, and turn-based game logic.

Together, the two programs reinforced important programming concepts including file handling, functions, loops, dictionaries, conditional logic, and data validation. The assignment also illustrated how Python can be used to build interactive applications that combine user input with external data sources. Overall, the project strengthened both programming skills and problem-solving abilities through the development of two complete game applications.